ICS 202 – Data Structures and Algorithms

Topic 08: AVL Trees

Question bank: 40 multiple choice + 10 programming

Questions from ICS 202 old exams, quizzes and activities (AI_Work). **[new]** marks new questions written in the same style.

Notation. A tree is written as a list *node: left, right* ("–" means no child; nodes not listed are leaves). Results are given in *level order*; inserting a level-order list into an empty BST rebuilds exactly that tree. Balance factor = height(right subtree) − height(left subtree), height(null) = -1, height(leaf) = 0.

Part A: Multiple choice

Q1.

Old exams compilation (T222)

Choose the true statements from the following: (i) An AVL tree is a self-balancing binary search tree. (ii) In any AVL tree, the heights of the two child subtrees of any node differ by at most one. (iii) In an AVL tree, when nodes are inserted in or deleted from the tree, if the AVL tree becomes unbalanced, repair the tree until balance is restored. (iv) Rebalancing operations of AVL tree maintains a balanced $O(\log n)$ height.

(A) None of the statements is true.

(B) All the statements are true.

(C) Only statements iii and iv are true.

(D) Only statements i and ii are true.

Answer: (B). An AVL tree is a height-balanced BST that is rebalanced after every insertion/deletion, which keeps its height $O(\log n)$.

My answer

Q2.

Quiz 3 Ver A (T251)

For an AVL tree with n nodes, the worst-case time complexity for searching an element is:

(A) $\Theta(\log n)$

(B) $\Theta(\sqrt{n})$

(C) $\Theta(\log^2 n)$

(D) $\Theta(n)$

Answer: (A). The height of an AVL tree is $O(\log n)$, and search follows one root-to-leaf path.

My answer

Q3. Activity 08-01 (T252)

Compute the balance factors of the following BST. Is it an AVL tree?
`33: 13, 53; 13: 11, 21; 11: 8, --; 8: --, 9; 53: --, 61`

(A) Yes, every node has a balance factor of -1, 0 or $+1$.

(B) No; node 11 is the only node with balance factor -2.

(C) No; nodes 33, 13 and 11 all have balance factor -2.

(D) No; node 8 has balance factor $+2$.

Answer: (C). Heights: 8 is 1, 11 is 2, 13 is 3, 53 is 1. So bf(11) $= -1 - 1 = -2$, bf(13) $= 0 - 2 = -2$, bf(33) $= 1 - 3 = -2$; bf(8) $= +1$.

My answer

Q4. Old exams compilation (T222); Quiz 3 V1 (T251)

Given the following three trees:
Tree 1: `8: 3, --; 3: 1, --`
Tree 2: root 4 with left child 2 and right child 5; node 2 has only a right child (key 4 in the figure), which has only a left child 3.
Tree 3: `14: 9, 28; 9: 6, --; 28: --, 31; 31: --, 44`

(A) All the three trees are not AVL trees.

(B) All the three trees are AVL trees.

(C) Only tree 1 is an AVL tree.

(D) Only trees 1 and 3 are AVL trees.

Answer: (A). Tree 1: bf(8) $= -2$. Tree 2: bf(2) $= +2$. Tree 3: bf(28) $= +2$.

My answer

Q5. Quiz 3 Ver A/B (T241)

Which of the following values, when inserted into an empty Binary Search Tree, will result in an AVL tree as well (without the need to carry out any rotation)?

(A) 1, 2, 3, 4, 5, 6, 7, 8

(B) 4, 3, 5, 1, 2

(C) 4, 3, 5, 1, 2, 6

(D) None of the given answers is correct.

Answer: (D). (A) gives a chain. In (B) and (C) node 3 has left subtree $1 \to 2$ (height 1) and no right child, so bf(3) $= -2$.

My answer

Q6. Quiz 3 Ver A (T241); Quiz 3 Ver A (T251)

In the following AVL tree, the balance factor of the node with key 42 is
`37: 24, 42; 24: 7, 32; 7: 2, --; 42: 40, 52; 52: --, 120`

(A) 0

(B) 1

(C) -1

(D) 2

Answer: (B). height(40) $= 0$, height(52) $= 1$, so bf(42) $= 1 - 0 = 1$.

My answer

Q7. Quiz 3 Ver B (T241)

Use the AVL tree `37: 24, 42; 24: 7, 32; 7: 2, --; 42: 40, 52; 52: --, 120`. The balance factor of key 24 is

(A) 0

(B) 1

(C) -1

(D) -2

Answer: (C). height(7) $= 1$, height(32) $= 0$, so bf(24) $= 0 - 1 = -1$.

My answer

Q8. Quiz 3 Ver A (T242)

Use the following AVL tree (the figure's left child of 12 is written here as 11.5 so that the tree is a valid BST):
`8: 5, 11; 5: 2, 6; 2: --, 4; 6: --, 7; 11: 10, 12; 10: 9, --; 12: 11.5, 14; 14: --, 15`
The balance factor of key 11 is

(A) 0

(B) 1

(C) -1

(D) 2

Answer: (B). height(10) $= 1$, height(12) $= 2$, so bf(11) $= 2 - 1 = 1$.

My answer

Q9. Quiz 3 Ver A/B (T241)

Once we have a reference to the deepest unbalanced node, the time complexity of carrying out a double rotation, without including the time complexity of adjusting the height, is

(A) $\Theta(1)$

(B) $\Theta(\log n)$

(C) $\Theta(n)$

(D) $\Theta(n \log n)$

Answer: (A). A double rotation is two single rotations, each changing a constant number of references.

My answer

Q10. Activity 08-01 (T252)

Identify the imbalanced node and perform the correct rotation for the tree
`43: 11, 65; 11: 8, 27; 8: 3, --`

(A) Single right rotation about 43; result (level order): 11, 8, 43, 3, 27, 65

(B) Single right rotation about 11; result: 43, 8, 65, 3, 11, 27

(C) Double left-right rotation about 43; result: 27, 11, 43, 8, 65, 3

(D) Single left rotation about 43; result: 65, 43, 11, 8, 27, 3

Answer: (A). bf(43) $= -2$ and bf(11) $= -1$: LL case. 11 becomes the root, 43 its right child, and 27 (BR) becomes the left child of 43.

My answer

Q11. Activity 08-01 (T252)

Identify the imbalanced node and perform the correct rotation for the tree
`43: 21, 65; 65: 51, 87; 87: 73, --`

(A) Double right-left rotation about 43; result (level order): 51, 43, 65, 21, 87, 73

(B) Single right rotation about 87; result: 43, 21, 73, 65, 87, 51

(C) Single left rotation about 43; result: 65, 43, 87, 21, 51, 73

(D) Single left rotation about 65; result: 43, 21, 87, 65, 73, 51

Answer: (C). bf(43) $= +2$, bf(65) $= +1$: RR case. 65 becomes the root, 43 its left child, and 51 (BL) becomes the right child of 43.

My answer

Q12. Activity 08-01 (T252)

For the tree
`20: 10, 50; 10: 8, 15; 15: 11, --; 50: 32, 60; 32: --, 46; 46: --, 48`
which node is the deepest unbalanced node, and which rotation rebalances the tree?

(A) 50; single right rotation

(B) 32; single left rotation

(C) 32; double right-left rotation

(D) 20; single left rotation

Answer: (B). bf(32) $= +2$ and bf(46) $= +1$ (RR), so rotate left about 32: subtree becomes 46 with children 32 and 48. (bf(50) $= -2$ too, but 32 is deeper.)

My answer

Q13. Activity 08-01 (T252)

After inserting 40, the tree is
`51: 16, 71; 16: 12, 23; 23: 22, 31; 31: --, 40; 71: 61, 82; 82: --, 98`
What is the tree (level order) after performing the correct rotation?

(A) 51, 16, 71, 12, 31, 61, 82, 23, 40, 98, 22

(B) 51, 23, 71, 16, 31, 61, 82, 12, 22, 40, 98

(C) 23, 16, 51, 12, 22, 31, 71, 40, 61, 82, 98

(D) 51, 22, 71, 16, 23, 61, 82, 12, 31, 98, 40

Answer: (B). Deepest unbalanced node is 16 (bf $= +2$, bf(23) $= +1$): RR, single left rotation about 16; 23 moves up and 22 becomes the right child of 16.

My answer

Q14. Activity 08A-02 (T252)

After inserting 56, the tree is `57: 52, 85; 52: 46, 55; 55: 53, 56`. Identify the imbalanced node and perform the correct rotation.

(A) Single right rotation about 57; result (level order): 52, 46, 57, 55, 85, 53, 56

(B) Double left-right rotation about 57; result: 55, 52, 57, 46, 53, 56, 85

(C) Single left rotation about 52; result: 57, 55, 85, 52, 56, 46, 53

(D) Double right-left rotation about 57; result: 53, 52, 57, 46, 55, 56, 85

Answer: (B). bf(57) $= -2$, bf(52) $= +1$: LR case. 55 becomes the root, 53 becomes the right child of 52, and 56 becomes the left child of 57.

My answer

Q15. Activity 08A-02 (T252)

After inserting 51, the tree is
`35: 9, 57; 9: 5, 23; 57: 42, 91; 42: 37, 47; 47: --, 51; 91: --, 96`
Which rotation rebalances it, and what is the new root?

(A) Single left rotation about 35; new root 57

(B) Double right-left rotation about 35; new root 42

(C) Single left rotation about 42; root stays 35

(D) Double left-right rotation about 57; new root 47

Answer: (B). Only 35 is unbalanced: bf(35) $= +2$, bf(57) $= -1$ (RL). Result: 42 root, 35 (children 9, 37) left, 57 (children 47, 91) right.

My answer

Q16. Quiz 3 Ver A (T241)

Use the AVL tree `37: 24, 42; 24: 7, 32; 7: 2, --; 42: 40, 52; 52: --, 120`. Insertion of which key value may lead to a single left rotation to rebalance the AVL tree?

(A) 130

(B) 62

(C) 5

(D) 1

Answer: (A). 130 goes right of 120: bf(52) $= +2$, bf(120) $= +1$ (RR). 62 gives RL at 52, 5 gives LR at 7, 1 gives LL at 7.

My answer

Q17. Quiz 3 Ver A (T241)

Use the same AVL tree `37: 24, 42; 24: 7, 32; 7: 2, --; 42: 40, 52; 52: --, 120`. Insertion of which key value will definitely force a double left-right rotation to rebalance the AVL tree?

(A) 130

(B) 62

(C) 5

(D) 1

Answer: (C). 5 becomes the right child of 2: bf(7) $= -2$, bf(2) $= +1$, the LR case.

My answer

Q18. Quiz 3 Ver B (T241)

Use the AVL tree `37: 24, 42; 24: 7, 32; 7: 2, --; 42: 40, 52; 52: --, 120`. Insertion of which key value may lead to a single right rotation to rebalance the AVL tree?

(A) 130

(B) 62

(C) 5

(D) 1

Answer: (D). 1 becomes the left child of 2: bf(7) $= -2$, bf(2) $= -1$ (LL), rotate right about 7.

My answer

Q19. Quiz 3 Ver B (T241)

Use the AVL tree `37: 24, 42; 24: 7, 32; 7: 2, --; 42: 40, 52; 52: --, 120`. Insertion of which key value will definitely force a double right-left rotation to rebalance the AVL tree?

(A) 130

(B) 62

(C) 5

(D) 1

Answer: (B). 62 becomes the left child of 120: bf(52) $= +2$, bf(120) $= -1$, the RL case.

My answer

Q20. Quiz 3 Ver A (T242)

Use the AVL tree
`8: 5, 11; 5: 2, 6; 2: --, 4; 6: --, 7; 11: 10, 12; 10: 9, --; 12: 11.5, 14; 14: --, 15`
Insertion of which key value may lead to a single right rotation only, to rebalance the AVL tree?

(A) 6.5

(B) 7.5

(C) 8.5

(D) 9.5

Answer: (C). 8.5 becomes the left child of 9: bf(10) $= -2$, bf(9) $= -1$ (LL). 6.5 gives RL at 6, 7.5 gives RR at 6, 9.5 gives LR at 10.

My answer

Q21. Old exams compilation (T131)

Consider the AVL tree
`30: 20, 45; 20: 11, 25; 11: 7, 15; 7: 5, --; 25: 22, --; 45: 38, 49; 49: 47, --`
The operation that will cause a double left-right rotation on the AVL tree is:

(A) inserting key 48

(B) inserting key 46

(C) deleting key 38

(D) deleting key 5

Answer: (A). 48 becomes the right child of 47: bf(49) $= -2$, bf(47) $= +1$ (LR). 46 gives LL at 49; deleting 38 gives RL at 45; deleting 5 needs no rotation.

My answer

Q22. Old exams compilation (T121)

Consider the same AVL tree
`30: 20, 45; 20: 11, 25; 11: 7, 15; 7: 5, --; 25: 22, --; 45: 38, 49; 49: 47, --`
The operation that may cause a single right rotation, without any double rotations on the AVL tree, is:

(A) inserting key 48

(B) deleting key 38

(C) deleting key 5

(D) none of the above

Answer: (D) (old key said deleting key 5). Deleting 5 leaves bf(7) $= 0$, bf(11) $= 0$, bf(20) $= 0$, bf(30) $= 0$: no rotation at all. Inserting 48 needs LR; deleting 38 needs RL at 45 (then a single right at 30).

My answer

Q23. Old exams compilation (T211)

Insert key 12 into the AVL tree `6: 3, 8; 3: 2, 5; 5: 4, --; 8: 7, 10; 10: --, 17`. Which rotation is needed and what replaces the subtree rooted at 10?

(A) Single left rotation; subtree `17: 10, --; 10: --, 12`

(B) Double right-left rotation; subtree `12: 10, 17`

(C) Single right rotation; subtree `12: 10, 17`

(D) Double left-right rotation; subtree `17: 12, --; 12: 10, --`

Answer: (B). 12 is the left child of 17: bf(10) $= +2$, bf(17) $= -1$ (RL). Final tree: 6, 3, 8, 2, 5, 7, 12, 4, 10, 17.

My answer

Q24. Old exams compilation (T211)

Now insert 20 into the resulting tree `6: 3, 8; 3: 2, 5; 5: 4, --; 8: 7, 12; 12: 10, 17`. The final AVL tree (level order) and rotation are:

(A) Single left about 8: 6, 3, 12, 2, 5, 8, 17, 4, 7, 10, 20

(B) Single left about 6: 8, 6, 12, 3, 7, 10, 17, 2, 5, 20, 4

(C) No rotation: 6, 3, 8, 2, 5, 7, 12, 4, 10, 17, 20

(D) Double right-left about 8: 6, 3, 10, 2, 5, 8, 12, 4, 7, 17, 20

Answer: (A). 20 goes right of 17: bf(8) $= +2$, bf(12) $= +1$ (RR). 12 moves up, 10 becomes the right child of 8.

My answer

Q25. Old exams compilation (T222); Activity 08A-02 (T252)

Given the AVL tree `12: 8, 16; 8: 4, 10; 4: 2, 6; 16: 14, --`, insert 5. The updated AVL tree (level order) is:

(A) 12, 8, 16, 4, 10, 14, 2, 6, 5

(B) 12, 4, 16, 2, 8, 14, 6, 10, 5

(C) 8, 4, 12, 2, 6, 10, 16, 5, 14

(D) 12, 6, 16, 4, 8, 14, 2, 5, 10

Answer: (D). 5 goes left of 6: bf(8) $= -2$, bf(4) $= +1$ (LR). Left rotation about 4, then right rotation about 8: 6 moves up, 5 becomes the right child of 4.

My answer

Q26. Activity 08A-02 (T252)

Insert 15 into the AVL tree `5: 2, 7; 2: 1, 4; 4: 3, --; 7: 6, 9; 9: --, 16` and balance the tree. The result (level order) is:

(A) 5, 2, 9, 1, 4, 7, 16, 3, 6, 15

(B) 5, 2, 7, 1, 4, 6, 15, 3, 9, 16

(C) 5, 2, 7, 1, 4, 6, 16, 3, 9, 15

(D) 7, 5, 9, 2, 6, 16, 1, 4, 15, 3

Answer: (B). 15 is the left child of 16: bf(9) $= +2$, bf(16) $= -1$ (RL). 15 replaces 9 with children 9 and 16.

My answer

Q27. Quiz 3 (T232); Quiz 3 V1 (T251)

Insert a node with key 47 into the AVL tree
`18: 9, 35; 9: 7, 15; 7: --, 8; 35: 27, 43; 27: --, 29; 43: 39, 48; 48: 46, --`
The resulting AVL tree (level order) is:

(A) 18, 9, 35, 7, 15, 27, 43, 8, 29, 39, 47, 46, 48

(B) 18, 9, 35, 7, 15, 27, 46, 8, 29, 43, 48, 39, 47

(C) 18, 9, 43, 7, 15, 35, 48, 8, 27, 39, 46, 29, 47

(D) 18, 9, 35, 7, 15, 27, 43, 8, 29, 39, 46, 48, 47

Answer: (A). 47 is the right child of 46: bf(48) $= -2$, bf(46) $= +1$ (LR at 48). 47 moves up with children 46 and 48.

My answer

Q28. Old exams compilation (T161)

Insert 2.5 into the AVL tree `8: 4, 12; 4: 2, 5; 2: --, 3; 5: --, 7; 12: 9, 14; 9: --, 10`. The resulting AVL tree (level order) is:

(A) 8, 4, 12, 3, 5, 9, 14, 2, 2.5, 7, 10

(B) 8, 3, 12, 2, 4, 9, 14, 2.5, 5, 10, 7

(C) 8, 4, 12, 2.5, 5, 9, 14, 2, 3, 7, 10

(D) 8, 4, 12, 2, 5, 9, 14, 3, 7, 10, 2.5

Answer: (C). 2.5 is the left child of 3: bf(2) $= +2$, bf(3) $= -1$ (RL at 2). 2.5 moves up with children 2 and 3.

My answer

Q29. Old exams compilation (T181)

Insert these keys into an empty AVL tree: 2, 5, 4, 9, 10, 12, 11. The final tree (level order) is:

(A) 4, 2, 10, 5, 11, 9, 12

(B) 9, 4, 11, 2, 5, 10, 12

(C) 9, 4, 10, 2, 5, 12, 11

(D) 5, 4, 10, 2, 9, 12, 11

Answer: (B). Rotations: RL at 2 (after 4), RR at 5 (after 10), RR at 4 (after 12, 9 becomes root), RL at 10 (after 11).

My answer

Q30. Old exams compilation (T131/T132)

Insert the following nodes into an empty AVL tree: 27, 28, 37, 10, 40, 35, 25, 60. The final tree (level order) is:

(A) 28, 25, 37, 10, 27, 35, 40, 60

(B) 28, 27, 37, 10, 35, 40, 25, 60

(C) 27, 25, 37, 10, 28, 35, 40, 60

(D) 28, 10, 37, 25, 27, 35, 40, 60

Answer: (A). Single left at 27 (after 37); after 25, bf(27) $= -2$ and bf(10) $= +1$: LR, so 25 moves up with children 10 and 27; 60 needs no rotation.

My answer

Q31. Old exams compilation (T131)

Insert the data values 5, 12, 20, 10, 15, 30, 26, 23, 25 into an empty AVL tree. The final tree (level order) is:

(A) 12, 5, 26, 10, 20, 30, 15, 23, 25

(B) 20, 12, 26, 5, 15, 23, 30, 10, 25

(C) 12, 5, 23, 10, 20, 26, 15, 25, 30

(D) 12, 5, 20, 10, 15, 26, 23, 30, 25

Answer: (C). Rotations: RR at 5 (after 20), LL at 30 (after 23), RL at 20 (after 25: 23 moves up with children 20 and 26).

My answer

Q32. Quiz 3 Ver A (T251)

Insert the keys 7, 8, 9, 10, 11, respectively, into an initially empty AVL tree. How many rotations are performed and what is the final tree (level order)?

(A) One single left rotation; 8, 7, 9, 10, 11

(B) Two single left rotations; 8, 7, 10, 9, 11

(C) Two single left rotations; 9, 8, 10, 7, 11

(D) One double right-left rotation; 9, 7, 10, 8, 11

Answer: (B). Left rotation about 7 after 9 (8 becomes root), left rotation about 9 after 11 (10 replaces 9).

My answer

Q33. Assignment 3 (T252)

Construct an AVL tree by inserting the keys 35, 20, 50, 10, 5, 25, 60, 70, in order, into an initially empty tree. Which sequence of rotation cases occurs?

(A) LL (single right), LR (double left-right), RR (single left)

(B) LL (single right), LL (single right), RR (single left)

(C) LR (double left-right), LL (single right), RR (single left)

(D) LL (single right), RL (double right-left), RR (single left)

Answer: (A). Insert 5: LL at 20. Insert 25: bf(35) $= -2$, bf(10) $= +1$, LR (20 becomes root). Insert 70: RR at 50. Final: 20, 10, 35, 5, 25, 60, 50, 70.

My answer

Q34. Quiz 3 Ver A (T241)

Use the AVL tree `37: 24, 42; 24: 7, 32; 7: 2, --; 42: 40, 52; 52: --, 120`. Deletion of key 37 (by copying) will

(A) not result in any rotations needed to rebalance the tree.

(B) result in a double right-left rotation to rebalance the tree.

(C) result in a double left-right rotation to rebalance the tree.

(D) result in a single left rotation or a single right rotation to rebalance the tree.

Answer: (D). With the predecessor 32, bf(24) becomes -2 with bf(7) $= -1$: single right. With the successor 40, bf(42) becomes $+2$ with bf(52) $= +1$: single left.

My answer

Q35. Quiz 3 Ver B (T241)

Use the AVL tree `37: 24, 42; 24: 7, 32; 7: 2, --; 42: 40, 52; 52: --, 120`. Deletion of key 42 (by copying) will

(A) result in a single left rotation or no rotation is needed to rebalance the AVL tree.

(B) not result in any rotations needed to rebalance the tree.

(C) result in a double right-left rotation to rebalance the tree.

(D) result in a double left-right rotation to rebalance the tree.

Answer: (A). Predecessor 40 replaces 42 and leaves bf $= +2$ with bf(52) $= +1$: single left. Successor 52 replaces 42 and gives 52 with children 40, 120: no rotation.

My answer

Q36. Quiz 3 Ver A (T242)

Use the AVL tree
`8: 5, 11; 5: 2, 6; 2: --, 4; 6: --, 7; 11: 10, 12; 10: 9, --; 12: 11.5, 14; 14: --, 15`
Deletion of which key value will not require any rotation to rebalance the AVL tree?

(A) 11.5

(B) 10

(C) 9

(D) 6

Answer: (D). 7 replaces 6, and the height of the subtree of 5 does not change. Deleting 11.5 makes bf(12) $= +2$; deleting 10 or 9 makes bf(11) $= +2$.

My answer

Q37. Old exams compilation (T142); Activity 08A-02 (T252)

Deleting 40 from the following AVL tree will result in how many single rotations?
`30: 20, 35; 20: 10, 25; 10: 5, 15; 5: 1, --; 25: 22, --; 35: 32, 40; 32: 31, --`

(A) 0

(B) 1

(C) 2

(D) 3

Answer: (C). First a right rotation about 35 (32 moves up); the subtree height drops, so bf(30) $= -2$ with bf(20) $= -1$: a second right rotation about 30. Final: 20, 10, 30, 5, 15, 25, 32, 1, 22, 31, 35.

My answer

Q38. Old exams compilation (T161)

Consider the AVL tree `8: 4, 12; 4: 2, 5; 2: --, 3; 5: --, 7; 12: 9, 14; 9: --, 10`. Delete 5, then delete 7. The resulting AVL tree (level order) is:

(A) 8, 4, 12, 2, 9, 14, 3, 10

(B) 8, 2, 12, 4, 9, 14, 3, 10

(C) 8, 3, 12, 2, 4, 9, 14, 10

(D) 9, 3, 12, 2, 8, 10, 14, 4

Answer: (C). After both deletions 4 has only left child 2 (which has right child 3): bf(4) $= -2$, bf(2) $= +1$ (LR). 3 moves up with children 2 and 4.

My answer

Q39.

Old exams compilation (T131)

Consider the AVL tree
`28: 15, 50; 15: 7, 22; 7: --, 10; 50: 39, 55; 39: 33, 42; 33: 30, --; 55: --, 63`
Delete the node with key 15 (deletion by copying, using the inorder predecessor). The resulting AVL tree (level order) is:

(A) 28, 10, 50, 7, 22, 39, 55, 33, 42, 63, 30

(B) 39, 28, 50, 10, 33, 42, 55, 7, 22, 30, 63

(C) 50, 28, 55, 10, 39, 63, 7, 22, 33, 42, 30

(D) 33, 28, 50, 10, 30, 39, 55, 7, 22, 42, 63

Answer: (B). 10 replaces 15, so the left subtree of 28 has height 1 and the right has height 3: bf(28) $= +2$, bf(50) $= -1$ (RL). 39 becomes the root; 33 becomes the right child of 28 and 42 the left child of 50.

My answer

Q40.

Activity 08A-02 (T252)

Delete 32 from the AVL tree `44: 17, 62; 17: --, 32; 62: 50, 78; 50: 48, 54; 78: --, 88` and do the necessary rotation. The result (level order) is:

(A) 50, 44, 62, 17, 48, 54, 78, 88

(B) 44, 17, 62, 50, 78, 48, 54, 88

(C) 62, 44, 78, 17, 50, 88, 48, 54

(D) 54, 44, 62, 17, 50, 78, 48, 88

Answer: (C). bf(44) $= +2$ and bf(62) $= 0$: single left rotation about 44; 50 (BL) becomes the right child of 44.

My answer

Part B: Programming

For Questions 1–3 and 9 use

```
class BTNode { int key; BTNode left, right; BTNode(int k) { key = k; } }
```

For Questions 4–8 and 10 use a node that stores its own height (a leaf has height 0, null has height -1):

```
class AVLNode {
    int key; int height; AVLNode left, right;
    AVLNode(int k) { key = k; height = 0; }
}
static int height(AVLNode n) { return n == null ? -1 : n.height; }
```

Q41. [new]

Write the recursive method `int getHeight(BTNode node)` used by the slides' `AVLTree` class: it returns -1 for an empty tree, 0 for a leaf, and $1 + \max$(height of left, height of right) otherwise.

Answer:

```
static int getHeight(BTNode node) {
    if (node == null) return -1;
    return 1 + Math.max(getHeight(node.left), getHeight(node.right));
}
```

My answer

Q42. [new]

Using `getHeight` from the previous question, write `int balanceFactor(BTNode node)` that returns height(right subtree) $-$ height(left subtree) (return 0 for null). For the tree `37: 24, 42; 24: 7, 32; 7: 2, --; 42: 40, 52; 52: --, 120` it must return $+1$ for 42 and -1 for 24.

Answer:

```
static int balanceFactor(BTNode node) {
    if (node == null) return 0;
    return getHeight(node.right) - getHeight(node.left);
}
```

My answer

Q43. [new]

Write `boolean isAVL(BTNode root)` that returns true if every node of the (binary search) tree has a balance factor of -1, 0 or $+1$. Your method must run in $O(n)$: do not call `getHeight` at every node. (Hint: a helper returns the height of a subtree, or -2 if the subtree is not balanced.)

Answer:

```
static boolean isAVL(BTNode root) {
    return checkHeight(root) != -2;
}
// returns the height of node, or -2 if some node in its subtree is unbalanced
private static int checkHeight(BTNode node) {
    if (node == null) return -1;
    int hl = checkHeight(node.left);
    if (hl == -2) return -2;
    int hr = checkHeight(node.right);
    if (hr == -2) return -2;
    if (Math.abs(hr - hl) > 1) return -2;
    return 1 + Math.max(hl, hr);
}
```

My answer

Q44. [new]

Write `void adjustHeight(AVLNode n)` that recomputes `n.height` from its children, and `AVLNode rotateRight(AVLNode a)` that performs a single right rotation (LL case) about `a` and returns the new root of the subtree: `a`'s left child `b` becomes the root, `a` becomes the right child of `b`, and `b`'s right subtree (BR) becomes the left child of `a`. Update the heights.

Answer:

```
static void adjustHeight(AVLNode n) {
    n.height = 1 + Math.max(height(n.left), height(n.right));
}
static AVLNode rotateRight(AVLNode a) {
    AVLNode b = a.left;
    a.left = b.right;   // BR becomes left child of A
    b.right = a;        // A becomes right child of B
    adjustHeight(a);    // A is now lower: adjust it first
    adjustHeight(b);
    return b;           // B becomes the root
}
```

My answer

Q45. [new]

Write `AVLNode rotateLeft(AVLNode a)` that performs a single left rotation (RR case) about `a`: the right child `b` of `a` becomes the root of the subtree, `a` becomes the left child of `b`, and the left subtree of `b` (BL) becomes the right child of `a`. Update the heights and return the new subtree root. You may use `adjustHeight`.

Answer:

```
static AVLNode rotateLeft(AVLNode a) {
    AVLNode b = a.right;
    a.right = b.left;   // BL becomes right child of A
    b.left = a;         // A becomes left child of B
    adjustHeight(a);
    adjustHeight(b);
    return b;
}
```

My answer

Q46. [new]

Using `rotateLeft` and `rotateRight`, write `rotateLeftRight(AVLNode a)` (LR case) and `rotateRightLeft(AVLNode a)` (RL case). As in the slides, the first pivot is the child of the deepest unbalanced node `a`, and the second pivot is `a` itself. Each returns the new subtree root.

Answer:

```
static AVLNode rotateLeftRight(AVLNode a) {
    a.left = rotateLeft(a.left);   // first pivot: left child of A
    return rotateRight(a);         // second pivot: A
```

```
}
static AVLNode rotateRightLeft(AVLNode a) {
    a.right = rotateRight(a.right); // first pivot: right child of A
    return rotateLeft(a);           // second pivot: A
}
```

My answer

Q47. [new]

Write `AVLNode rebalance(AVLNode n)`. It first adjusts the height of `n`, computes its balance factor (right − left) and, if it is -2 or $+2$, applies the correct case (LL, LR, RR or RL) using the rotation methods above. It returns the root of the (possibly new) subtree. (Treat a child balance factor of 0 as the single-rotation case; this can happen after deletion.)

Answer:

```
static int bf(AVLNode n) { return height(n.right) - height(n.left); }

static AVLNode rebalance(AVLNode n) {
    adjustHeight(n);
    int b = bf(n);
    if (b == -2) {
        if (bf(n.left) <= 0) return rotateRight(n);    // LL
        else return rotateLeftRight(n);                // LR
    } else if (b == 2) {
        if (bf(n.right) >= 0) return rotateLeft(n);    // RR
        else return rotateRightLeft(n);                // RL
    }
    return n;
}
```

My answer

Q48. [new]

Write the recursive method `AVLNode insert(AVLNode node, int key)` that inserts `key` with the BST insertion algorithm and rebalances every node on the path back to the root (use `rebalance`). Ignore duplicates. Inserting 40, 20, 10, 25, 30, 22, 50 into an empty tree must give the slides' tree 25, 20, 40, 10, 22, 30, 50 (level order).

Answer:

```
static AVLNode insert(AVLNode node, int key) {
    if (node == null) return new AVLNode(key);
    if (key < node.key) node.left = insert(node.left, key);
    else if (key > node.key) node.right = insert(node.right, key);
    else return node;            // no duplicates
    return rebalance(node);
}
// usage: root = insert(root, key);
```

My answer

Q49. [new]

A key has just been inserted into a BST with the ordinary BST insertion (no rebalancing). Write `BTNode deepestUnbalanced(BTNode root, int key)` that walks the path from the root to the node containing `key` and returns the deepest node on that path whose balance factor is -2 or $+2$ (this is the pivot of the rotation), or null if there is none. Use `balanceFactor` from Question 2. Example: after inserting 130 into `37: 24, 42; 24: 7, 32; 7: 2, --; 42: 40, 52; 52: --, 120` the method returns the node 52.

Answer:

```
static BTNode deepestUnbalanced(BTNode root, int key) {
    BTNode deepest = null;
    BTNode p = root;
    while (p != null) {
        if (Math.abs(balanceFactor(p)) > 1) deepest = p;
        if (key == p.key) break;
        p = (key < p.key) ? p.left : p.right;
    }
    return deepest;
}
```

My answer

Q50. [new]

Write the recursive method `AVLNode delete(AVLNode node, int key)` for an AVL tree. Use deletion by copying: a node with two children takes the key of its inorder predecessor (maximum of its left subtree), and that predecessor is then deleted from the left subtree. Rebalance every node on the way back up (use `rebalance`); note that a deletion may need more than one rotation. Deleting 40 from `30: 20, 35; 20: 10, 25; 10: 5, 15; 5: 1, --; 25: 22, --; 35: 32, 40; 32: 31, --` must give 20, 10, 30, 5, 15, 25, 32, 1, 22, 31, 35.

Answer:

```
static AVLNode delete(AVLNode node, int key) {
    if (node == null) return null;               // key not found
    if (key < node.key) node.left = delete(node.left, key);
    else if (key > node.key) node.right = delete(node.right, key);
    else {
        if (node.left == null) return node.right;
        if (node.right == null) return node.left;
        AVLNode pred = node.left;                // inorder predecessor
        while (pred.right != null) pred = pred.right;
        node.key = pred.key;                     // copy it
        node.left = delete(node.left, pred.key); // delete the copy
    }
    return rebalance(node);
}
```

My answer